

JSON Web Token

JSON Web Token (**JWT**, suggested pronunciation /dʒɒt/, same as the word "jot")^[1] is a proposed Internet standard for creating data with optional signature and/or optional encryption whose payload holds JSON that asserts some number of claims. The tokens are signed either using a private secret or a public/private key.

For example, a server could generate a token that has the claim "logged in as administrator" and provide that to a client. The client could then use that token to prove that it is logged in as admin. The tokens can be signed by one party's private key (usually the server's) so that any party can subsequently verify whether the token is legitimate. If the other party, by some suitable and trustworthy means, is in possession of the corresponding public key, they too are able to verify the token's legitimacy. The tokens are designed to be compact,^[2] URL-safe,^[3] and usable, especially in a web-browser single-sign-on (SSO) context. JWT claims can typically be used to pass identity of authenticated users between an identity provider and a service provider, or any other type of claims as required by business processes.^{[4][5]}

JWT relies on other JSON-based standards: JSON Web Signature and JSON Web Encryption.^{[1][6][7]}

JSON Web Token

Abbreviation	JWT
Status	<u>Proposed Standard</u>
First published	December 28, 2010
Latest version	<u>RFC 7519</u> (https://www.rfc-editor.org/rfc/rfc7519) May 2015
Organization	<u>IETF</u>
Committee	<u>IEGS</u>
Authors	Michael B. Jones <u>Microsoft</u> John Bradley <u>Ping Identity</u> Nat Sakimura <u>NRI</u>
Base standards	<u>JSON</u> <u>JSON Web Encryption</u> (JWE) <u>JSON Web Signature</u> (JWS)
Domain	<u>Data exchange</u>
Website	datatracker.ietf.org/doc/html/rfc7519 (https://datatracker.ietf.org/doc/html/rfc7519)

Structure

Header

Identifies which algorithm is used to generate the signature. In the below example, HS256 indicates that this token is signed using HMAC-SHA256.

Typical cryptographic algorithms used are HMAC with SHA-256 (HS256) and RSA signature with SHA-256 (RS256). JWA (JSON Web Algorithms) RFC 7518 introduces many more for both authentication and encryption.^[8]

```
{  
  "alg": "HS256",  
}
```

```

{
  "typ": "JWT"
}

```

Payload

Contains a set of claims. The JWT specification defines seven Registered Claim Names, which are the standard fields commonly included in tokens.^[1] Custom claims are usually also included, depending on the purpose of the token.

This example has the standard Issued At Time claim (`iat`) and a custom claim (`loggedInAs`).

```
{
  "loggedInAs": "admin",
  "iat": 1422779638
}
```

Signature

Securely validates the token. The signature is calculated by encoding the header and payload using Base64url Encoding RFC 4648 (<https://www.rfc-editor.org/rfc/rfc4648>) and concatenating the two together with a period separator. That string is then run through the cryptographic algorithm specified in the header. This example uses HMAC-SHA256 with a shared secret (public key algorithms are also defined). The *Base64url Encoding* is similar to base64, but uses different non-alphanumeric characters and omits padding.

```
HMAC_SHA256(
    secret,
    base64urlEncoding(header) + '.' +
    base64urlEncoding(payload)
)
```

The three are encoded separately using Base64url Encoding RFC 4648 (<https://www.rfc-editor.org/rfc/rfc4648>), and concatenated using periods to produce the JWT:

```
const token: string = base64urlEncoding(header) + '.' + base64urlEncoding(payload) + '.' +
base64urlEncoding(signature)
```

The above data and the secret of "secretkey" creates the token:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJsb2dnZWRRJbkFzIjoieWRtaW4iLCJpYXQiOiE0MjI3Nzk2Mzh9LmZSraS8EXBxLN_oWnFSRgCzcmJmMjLiuyu5CSpyHI
```

(The above json strings are formatted without newlines or spaces, into utf-8 byte arrays. This is important as even slight changes in the data will affect the resulting token)

This resulting token can be easily passed into HTML and HTTP.^[3]

Use

In authentication, when a user successfully logs in, a JSON Web Token (JWT) is often returned. This token should be sent to the client using a secure mechanism like an HTTP-only cookie. Storing the JWT locally in browser storage mechanisms like local or session storage is discouraged. This is because

JavaScript running on the client-side (including browser extensions) can access these storage mechanisms, exposing the JWT and compromising security. To make use of the HTTP-only cookie, as you might need it to authenticate with cross-origin APIs, the best approach is to use the `credentials` property to tell the browser to automatically send the cookies to the external APIs via a Fetch call like so:

```
fetch('https://api.example.com/data', {
  method: 'GET',
  credentials: 'include' // This tells the browser to include cookies, etc.
})
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

By using this method, the JWT is never exposed to client-side JavaScript; this is the best approach to make use of your JWT while maintaining security best practices. For unattended processes, the client may also authenticate directly by generating and signing its own JWT with a pre-shared secret and passing it to an OAuth compliant service like so:

```
POST /oauth2/token
Content-type: application/x-www-form-urlencoded

grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer&assertion=eyJhb...
```

If the client passes a valid JWT assertion the server will generate an `access_token` valid for making calls to the application and pass it back to the client:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNTE2MjM5MDYyfQ%3D",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

When the client wants to access a protected route or resource, the user agent should send the JWT, typically in the **Authorization** HTTP header using the **Bearer** schema. The content of the header might look like the following:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ0In...

This is a stateless authentication mechanism as the user state is never saved in server memory. The server's protected routes will check for a valid JWT in the Authorization header, and if it is present, the user will be allowed to access protected resources. As JWTs are self-contained, all the necessary information is there, reducing the need to query the database multiple times.

Standard fields

Code	Name	Description
Standard claim fields		The internet drafts define the following standard fields ("claims") that can be used inside a JWT claim set.
iss	Issuer	Identifies principal that issued the JWT, e.g. the name of an organization or a URL of a website.
sub	Subject	Identifies the subject of the JWT, e.g. a username or account number.
aud	Audience	Identifies the recipients that the JWT is intended for. Each principal intended to process the JWT must identify itself with a value in the audience claim. If the principal processing the claim does not identify itself with a value in the aud claim when this claim is present, then the JWT must be rejected.
exp	Expiration Time	Identifies the expiration time on and after which the JWT must not be accepted for processing. The value must be a NumericDate: ^[9] either an integer or decimal, representing seconds past <u>1970-01-01 00:00:00Z</u> .
nbf	Not Before	Identifies the time on which the JWT will start to be accepted for processing. The value must be a NumericDate.
iat	Issued at	Identifies the time at which the JWT was issued. The value must be a NumericDate.
jti	JWT ID	Case-sensitive unique identifier of the token even among different issuers.
Commonly-used header fields		The following fields are commonly used in the header of a JWT
typ	Token type	If present, it must be set to a registered IANA Media Type (https://www.iana.org/assignments/media-types/media-types.xhtml).
cty	Content type	If nested signing or encryption is employed, it is recommended to set this to JWT; otherwise, omit this field. ^[1]
alg	Message authentication code algorithm	The issuer can freely set an algorithm to verify the signature on the token. However, some supported algorithms are insecure. ^[10]
kid	Key ID	A hint indicating which key the client used to generate the token signature. The server will match this value to a key on file in order to verify that the signature is valid and the token is authentic.
x5c	x.509 Certificate Chain	A certificate chain in RFC4945 format corresponding to the private key used to generate the token signature. The server will use this information to verify that the signature is valid and the token is authentic.
x5u	x.509 Certificate Chain URL	A URL where the server can retrieve a certificate chain corresponding to the private key used to generate the token signature. The server will retrieve and use this information to verify that the signature is authentic.
crit	Critical	A list of headers that must be understood by the server in order to accept the token as valid
Code	Name	Description

List of currently registered claim names can be obtained from [IANA JSON Web Token Claims Registry](#).^[11]

Implementations

JWT implementations exist for many languages and frameworks, including but not limited to:

- [.NET \(C# VB.Net etc.\)](#)^[12]
- [C](#)^[13]
- [C++](#)^{[14][15]}
- [Clojure](#)^[16]
- [Common Lisp](#)^[17]
- [Dart](#)^[18]
- [Elixir](#)^[19]
- [Erlang](#)
- [Go](#)^[20]
- [Haskell](#)^[21]
- [Java](#)^[22]
- [JavaScript](#)^[23]
- [Julia](#)^[24]
- [Lua](#)^[25]
- [Node.js](#)^[26]
- [OCaml](#)^[27]
- [Perl](#)^[28]
- [PHP](#)^[29]
- [PL/SQL](#)^[30]
- [PowerShell](#)^[31]
- [Python](#)^[32]
- [Racket](#)^[33]
- [Raku](#)^[34]
- [Ruby](#)^[35]
- [Rust](#)^{[36][37]}
- [Scala](#)^[38]
- [Swift](#)^[39]

Vulnerabilities

JSON web tokens may contain session state. But if project requirements allow session invalidation before JWT expiration, services can no longer trust token assertions by the token alone. To validate that the session stored in the token is not revoked, token assertions must be checked against a [data store](#). This renders the tokens no longer stateless, undermining the primary advantage of JWTs.^[40]

Security consultant Tim McLean reported vulnerabilities in some JWT libraries that used the `alg` field to incorrectly validate tokens, most commonly by accepting a `alg=none` token. While these vulnerabilities were patched, McLean suggested deprecating the `alg` field altogether to prevent similar implementation confusion.^[10] Still, new `alg=none` vulnerabilities are still being found in the wild, with four [CVEs](#) filed in the 2018-2021 period having this cause.^[41]

With proper design, developers can address algorithm vulnerabilities by taking precautions:^{[42][43]}

1. Never let the JWT header alone drive verification
2. Know the algorithms (avoid depending on the `alg` field alone)
3. Use an appropriate key size

Several JWT libraries were found to be vulnerable to an [invalid Elliptic-curve attack](#) in 2017.^[44]

Some have argued that JSON web tokens are difficult to use securely due to the many different encryption algorithms and options available in the standard, and that alternate standards should be used instead for both web frontends^[45] and backends.^[46]

See also

- [API key](#)
- [Access token](#)
- [Basic access authentication](#)
- [Digest access authentication](#)
- [Claims-based identity](#)
- [HTTP header](#)
- [Concise Binary Object Representation \(CBOR\)](#)

References

1. Jones, Michael B.; Bradley, John; Sakimura, Nat (May 2015). *JSON Web Token (JWT)* (<https://www.rfc-editor.org/rfc/rfc7519>). IETF. doi:10.17487/RFC7519 (<https://doi.org/10.17487%2FRFC7519>). ISSN 2070-1721 (<https://search.worldcat.org/issn/2070-1721>). RFC 7519 (<https://datatracker.ietf.org/doc/html/rfc7519>).
2. Nickel, Jochen (2016). *Mastering Identity and Access Management with Microsoft Azure* (<https://books.google.com/books?id=Q4dcDgAAQBAJ&pg=PA84>). Packt Publishing. p. 84. ISBN 9781785887888. Retrieved July 20, 2018.
3. "JWT.IO - JSON Web Tokens Introduction" (<https://jwt.io/introduction/>). *jwt.io*. Retrieved July 20, 2018.
4. Sevilleja, Chris. "The Anatomy of a JSON Web Token" (<https://scotch.io/tutorials/the-anatomy-of-a-json-web-token>). Retrieved May 8, 2015.
5. "Atlassian Connect Documentation" (<https://web.archive.org/web/20150518082002/https://developer.atlassian.com/static/connect/docs/latest/concepts/understanding-jwt.html>). *developer.atlassian.com*. Archived from the original (<https://developer.atlassian.com/static/connect/docs/latest/concepts/understanding-jwt.html>) on May 18, 2015. Retrieved May 8, 2015.
6. Jones, Michael B.; Bradley, John; Sakimura, Nat (May 2015). "draft-ietf-jose-json-web-signature-41 - JSON Web Signature (JWS)" (<https://tools.ietf.org/html/draft-ietf-jose-json-web-signature-41>). *tools.ietf.org*. Retrieved May 8, 2015.
7. Jones, Michael B.; Hildebrand, Joe (May 2015). "draft-ietf-jose-json-web-encryption-40 - JSON Web Encryption (JWE)" (<https://tools.ietf.org/html/draft-ietf-jose-json-web-encryption-40>). *tools.ietf.org*. Retrieved May 8, 2015.
8. Jones, Michael B. (May 2015). "draft-ietf-jose-json-web-algorithms-40 - JSON Web Algorithms (JWA)" (<https://tools.ietf.org/html/draft-ietf-jose-json-web-algorithms-40>). *tools.ietf.org*. Retrieved May 8, 2015.
9. Jones, Michael B.; Bradley, Bradley; Sakimura, Sakimura (May 2015). "'exp' (Expiration Time) Claim" (<https://datatracker.ietf.org/doc/html/rfc7519#section-4.1.4>). *JSON Web Token (JWT)* (<https://www.rfc-editor.org/rfc/rfc7519>). IETF. sec. 4.1.4. doi:10.17487/RFC7519 (<https://doi.org/10.17487%2FRFC7519>). ISSN 2070-1721 (<https://search.worldcat.org/issn/2070-1721>). RFC 7519 (<https://datatracker.ietf.org/doc/html/rfc7519>).
10. McLean, Tim (March 31, 2015). "Critical vulnerabilities in JSON Web Token libraries" (<https://www.chosenplaintext.ca/2015/03/31/jwt-algorithm-confusion.html>). Auth0. Retrieved March 29, 2016.
11. "JSON Web Token (JWT)" (<https://www.iana.org/assignments/jwt/jwt.xhtml/>). IANA. January 23, 2015. Retrieved December 5, 2024.
12. *jwt-dotnet* (<https://github.com/jwt-dotnet/jwt>) on github.com

13. [libjwt \(https://github.com/benmcollins/libjwt\)](https://github.com/benmcollins/libjwt) on github.com
14. [jwt-cpp \(https://github.com/Thalhammer/jwt-cpp\)](https://github.com/Thalhammer/jwt-cpp) on github.com
15. [POCO C++ Libraries library Poco : : JWT](#)
16. ["liquidz/clj-jwt" \(https://github.com/liquidz/clj-jwt\)](https://github.com/liquidz/clj-jwt). *GitHub*. Retrieved May 7, 2018.
17. [cljwt \(https://github.com/gschjetne/cljwt\)](https://github.com/gschjetne/cljwt) on github.com
18. [JustJWT \(https://github.com/deftomat/JustJWT\)](https://github.com/deftomat/JustJWT) on github.com
19. ["bryanjos/joken" \(https://github.com/bryanjos/joken\)](https://github.com/bryanjos/joken). *GitHub*. Retrieved May 7, 2018.
20. ["golang-jwt/jwt" \(https://github.com/golang-jwt/jwt\)](https://github.com/golang-jwt/jwt). *GitHub*. Retrieved January 8, 2018.
21. ["jose: JSON Object Signing and Encryption \(JOSE\) and JSON Web Token \(JWT\) library" \(https://hackage.haskell.org/package/jose\)](https://hackage.haskell.org/package/jose). *Hackage*. Retrieved December 25, 2022.
22. [auth0/java-jwt \(https://github.com/auth0/java-jwt\)](https://github.com/auth0/java-jwt) on github.com
23. ["kjur/jsrsasign" \(https://github.com/kjur/jsrsasign\)](https://github.com/kjur/jsrsasign). *GitHub*. Retrieved May 7, 2018.
24. ["JWTs.jl" \(https://github.com/JuliaWeb/JWTs.jl\)](https://github.com/JuliaWeb/JWTs.jl). *GitHub*. Retrieved October 31, 2025.
25. ["SkyLothar/lua-resty-jwt" \(https://github.com/SkyLothar/lua-resty-jwt\)](https://github.com/SkyLothar/lua-resty-jwt). *GitHub*. Retrieved May 7, 2018.
26. ["jsonwebtoken" \(https://www.npmjs.com/package/jsonwebtoken\)](https://www.npmjs.com/package/jsonwebtoken). *npm*. Retrieved May 7, 2018.
27. [ocaml-jwt \(https://github.com/besport/ocaml-jwt\)](https://github.com/besport/ocaml-jwt) on github.com
28. [Crypt::JWT \(https://metacpan.org/pod/Crypt::JWT\)](https://metacpan.org/pod/Crypt::JWT) on cpan.org
29. [lcobucci/jwt \(https://github.com/lcobucci/jwt\)](https://github.com/lcobucci/jwt) on github.com
30. Egan, Morten (February 7, 2019), *GitHub - morten-egan/jwt_ninja: PLSQL Implementation of JSON Web Tokens*. (https://github.com/morten-egan/jwt_ninja), retrieved March 14, 2019
31. ["SP3269/posh-jwt" \(https://github.com/SP3269/posh-jwt\)](https://github.com/SP3269/posh-jwt). *GitHub*. Retrieved August 1, 2018.
32. ["jpadilla/pyjwt" \(https://github.com/jpadilla/pyjwt\)](https://github.com/jpadilla/pyjwt). *GitHub*. Retrieved March 21, 2017.
33. [net-jwt \(https://pkgs.racket-lang.org/package/net-jwt\)](https://pkgs.racket-lang.org/package/net-jwt) on pkgs.racket-lang.org
34. [JSON-WebToken \(https://github.com/jamesalbert/JSON-WebToken\)](https://github.com/jamesalbert/JSON-WebToken) on github.com
35. [ruby-jwt \(https://github.com/jwt/ruby-jwt\)](https://github.com/jwt/ruby-jwt) on github.com
36. [jsonwebtoken \(https://github.com/Keats/jsonwebtoken\)](https://github.com/Keats/jsonwebtoken) on github.com
37. [rust-jwt \(https://github.com/mikkyang/rust-jwt\)](https://github.com/mikkyang/rust-jwt) on github.com
38. [jwt-scala \(https://github.com/pauldijou/jwt-scala\)](https://github.com/pauldijou/jwt-scala) on github.com
39. [\[1\]](#) on github.com
40. Slootweg, Sven. ["Stop using JWT for sessions" \(http://crypto.net/~joepie91/blog/2016/06/13/top-using-jwt-for-sessions/\)](http://crypto.net/~joepie91/blog/2016/06/13/top-using-jwt-for-sessions/). *joepie91 Ramblings*. Retrieved August 1, 2018.
41. ["CVE - Search Results" \(https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=jwt+none\)](https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=jwt+none). *cve.mitre.org*.
42. ["Common JWT security vulnerabilities and how to avoid them" \(https://connect2id.com/products/nimbus-jose-jwt/vulnerabilities\)](https://connect2id.com/products/nimbus-jose-jwt/vulnerabilities). Retrieved May 14, 2018.
43. Andreas, Happe. ["JWT: Signature vs MAC attacks" \(https://snikt.net/blog/2019/05/16/jwt-signature-vs-mac-attacks/\)](https://snikt.net/blog/2019/05/16/jwt-signature-vs-mac-attacks/). *snikt.net*. Retrieved May 27, 2019.
44. ["Critical Vulnerability in JSON Web Encryption" \(https://auth0.com/blog/critical-vulnerability-in-json-web-encryption/\)](https://auth0.com/blog/critical-vulnerability-in-json-web-encryption/). *Auth0 - Blog*. Retrieved October 14, 2023.
45. ["No Way, JOSE! Javascript Object Signing and Encryption is a Bad Standard That Everyone Should Avoid - Paragon Initiative Enterprises Blog" \(https://paragonie.com/blog/2017/03/json-web-tokens-is-bad-standard-that-everyone-should-avoid\)](https://paragonie.com/blog/2017/03/json-web-tokens-is-bad-standard-that-everyone-should-avoid). *paragonie.com*. Retrieved October 13, 2023.
46. ["Pitfalls of JWT Authorization" \(https://authzed.com/blog/pitfalls-of-jwt-authorization\)](https://authzed.com/blog/pitfalls-of-jwt-authorization). *authzed.com*. Retrieved November 16, 2023.

- [RFC 7519 \(https://www.rfc-editor.org/rfc/rfc7519\)](https://www.rfc-editor.org/rfc/rfc7519)
 - [jwt.io \(https://jwt.io/\)](https://jwt.io/) – specialized website about JWT with tools and documentation, maintained by Auth0
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=JSON_Web_Token&oldid=1363115009"